

Part 9 - Improve the Code

1. Decompose `AuthService.register_user` and remove `print()`-based logging

File: `eduplatform/services/auth_service.py`

Original

```
from typing import Optional, Dict, Any, Type, Union
from datetime import datetime, timedelta
import jwt
...

class AuthService:
    def __init__(self, user_repository, notification_repository, jwt_secret, jwt_expire_hours=24):
        ...

    def register_user(self, user_type, full_name, email, password, **kwargs):
        print(f"[AUTH] register_user called with: {user_type}, {full_name}, {email}, {kwargs}")
        print(f"[AUTH] Checking if email exists: {email}")
        if self.user_repo.email_exists(email):
            print(f"[AUTH] Email {email} already exists")
            raise ValueError("Email already registered")
        print(f"[AUTH] Email {email} is available")
        print(f"[AUTH] Creating user of type: {user_type.__name__}")
        try:
            if user_type == Student:
                if 'grade' not in kwargs:
                    raise ValueError("Grade is required for student registration")
                user = Student(full_name, email, password, kwargs['grade'])
                print(f"[AUTH] Created Student: {user}")
            elif user_type == Teacher:
                user = Teacher(full_name, email, password)
                print(f"[AUTH] Created Teacher: {user}")
            elif user_type == Parent:
                user = Parent(full_name, email, password)
                print(f"[AUTH] Created Parent: {user}")
            elif user_type == Admin:
                user = Admin(full_name, email, password)
                print(f"[AUTH] Created Admin: {user}")
            else:
```

```

        raise ValueError(f"Invalid user type: {user_type.__name__}")
    except Exception as e:
        print(f"[AUTH] Error creating user: {str(e)}")
        print(f"[AUTH] Exception type: {type(e).__name__}")
        raise
    print("[AUTH] Adding additional profile information")
    if 'phone' in kwargs:
        user._phone = kwargs['phone']
        print(f"[AUTH] Set phone: {kwargs['phone']}")
    if 'address' in kwargs:
        user._address = kwargs['address']
        print(f"[AUTH] Set address: {kwargs['address']}")
    print("[AUTH] Adding user to repository")
    try:
        self.user_repo.add(user)
        print("[AUTH] User added to repository successfully")
    except Exception as e:
        print(f"[AUTH] Error adding user to repository: {str(e)}")
        print(f"[AUTH] Exception type: {type(e).__name__}")
        raise
    print("[AUTH] Sending welcome notification")
    welcome_message = f"Welcome to EduPlatform, {full_name}! Your account has been
successfully created."
    try:
        notification = self.notification_repo.create_notification(
            recipient_id=user._id, title="Welcome to EduPlatform",
            message=welcome_message, notification_type="system"
        )
        print(f"[AUTH] Created notification: {notification}")
    except Exception as e:
        print(f"[AUTH] Error creating notification: {str(e)}")
        print(f"[AUTH] Exception type: {type(e).__name__}")
    token = self._generate_token(user)
    return {'user': user, 'token': token, 'user_type': self.user_repo.get_user_type(user)}

```

Improved

```

import logging
import secrets
from typing import Optional, Dict, Any, Type, Union
from datetime import datetime, timedelta
import jwt
...

```

```
logger = logging.getLogger(__name__)
```

```
class AuthService:
```

```
    def __init__(self, user_repository, notification_repository, jwt_secret, jwt_expire_hours=24):
```

```
        ...
```

```
        self._reset_tokens: Dict[str, Dict[str, Any]] = {} # used by fix #2 below
```

```
    def register_user(self, user_type, full_name, email, password, **kwargs):
```

```
        logger.info("Registering new %s (email=%s)", user_type.__name__, email)
```

```
        if self.user_repo.email_exists(email):
```

```
            raise ValueError("Email already registered")
```

```
        user = self._create_user_instance(user_type, full_name, email, password, kwargs)
```

```
        self._apply_optional_profile_fields(user, kwargs)
```

```
        self.user_repo.add(user)
```

```
        logger.info("User %s persisted successfully", user._id)
```

```
        self._send_welcome_notification(user, full_name)
```

```
        token = self._generate_token(user)
```

```
        return {'user': user, 'token': token, 'user_type': self.user_repo.get_user_type(user)}
```

```
    def _create_user_instance(self, user_type, full_name, email, password, kwargs):
```

```
        """Instantiate the correct User subclass for the requested user_type."""
```

```
        if user_type == Student:
```

```
            if 'grade' not in kwargs:
```

```
                raise ValueError("Grade is required for student registration")
```

```
            return Student(full_name, email, password, kwargs['grade'])
```

```
        if user_type == Teacher:
```

```
            return Teacher(full_name, email, password)
```

```
        if user_type == Parent:
```

```
            return Parent(full_name, email, password)
```

```
        if user_type == Admin:
```

```
            return Admin(full_name, email, password)
```

```
        raise ValueError(f"Invalid user type: {user_type.__name__}")
```

```
    def _apply_optional_profile_fields(self, user, kwargs):
```

```
        """Copy optional profile fields (phone, address) onto a newly created user."""
```

```
        if 'phone' in kwargs:
```

```
            user._phone = kwargs['phone']
```

```
        if 'address' in kwargs:
```

```
            user._address = kwargs['address']
```

```

def _send_welcome_notification(self, user, full_name):
    """Best-effort welcome notification: failure here must not undo registration."""
    welcome_message = f"Welcome to EduPlatform, {full_name}! Your account has been
successfully created."
    try:
        self.notification_repo.create_notification(
            recipient_id=user._id, title="Welcome to EduPlatform",
            message=welcome_message, notification_type="system"
        )
    except Exception:
        logger.exception("Failed to send welcome notification to user %s", user._id)

```

Why it's better: The original method mixed five unrelated concerns (validation, type dispatch, persistence, notification, and logging) in ~55 statements, which is exactly what Pylint's `R0915 too-many-statements` had already flagged (53/50) and what Part 7's manual review independently called out. Splitting it into `_create_user_instance`, `_apply_optional_profile_fields`, and `_send_welcome_notification` gives each piece one job and one reason to change, and each is independently testable. Replacing every `print()` with `logging` calls removes debug output that could leak PII (name/email/phone/address/kwags) to an uncontrolled stream, and lets the log level be configured per environment instead of being unconditionally on. The bare `except Exception as e: print(...); raise` around user creation/persistence added no value (it just re-raised) and is gone entirely; the notification's `except Exception` is now the only one left, on purpose, because that failure is genuinely meant to be swallowed but it's now logged via `logger.exception` instead of silently discarded, so the failure is at least visible to operators.

2. Fix the broken/insecure password-reset flow

File: `eduplatform/services/auth_service.py`

Original

```

def reset_password_request(self, email: str) -> bool:
    user = self.user_repo.get_by_email(email)
    if user:
        reset_token = f"reset_{len(str(hash(str(datetime.now()))))[-10:]}"
        reset_url = f"https://eduplatform.example.com/reset-password?token={reset_token}"
        self.notification_repo.create_notification(
            recipient_id=email, title="Password Reset Request",
            message=f"Click the following link to reset your password: {reset_url}",

```

```

        notification_type="system",
        metadata={'reset_token': reset_token,
                  'expires_at': (datetime.now() + timedelta(hours=1)).isoformat()}
    )
    return True

```

```

def reset_password(self, token: str, new_password: str) -> bool:
    if not token.startswith('reset_'):
        return False
    user_email = token.split('_')[-1]      # never actually contains the email!
    user = self.user_repo.get_by_email(user_email)
    if not user:
        return False
    user._password_hash, user._salt = user._hash_password(new_password)
    self.user_repo.update(user)

```

Improved

PASSWORD_RESET_TOKEN_TTL = timedelta(hours=1) # module-level constant

```

def reset_password_request(self, email: str) -> bool:
    user = self.user_repo.get_by_email(email)
    if user:
        reset_token = secrets.token_urlsafe(32)
        expires_at = datetime.now() + PASSWORD_RESET_TOKEN_TTL
        self._reset_tokens[reset_token] = {'email': email, 'expires_at': expires_at}

        reset_url = f"https://eduplatform.example.com/reset-password?token={reset_token}"
        self.notification_repo.create_notification(
            recipient_id=email, title="Password Reset Request",
            message=f"Click the following link to reset your password: {reset_url}",
            notification_type="system",
            metadata={'expires_at': expires_at.isoformat()}
        )
    return True

```

```

def reset_password(self, token: str, new_password: str) -> bool:
    token_data = self._reset_tokens.get(token)
    if not token_data or datetime.now() > token_data['expires_at']:
        self._reset_tokens.pop(token, None)
        return False

    user = self.user_repo.get_by_email(token_data['email'])
    del self._reset_tokens[token]    # single-use, regardless of outcome

```

```

if not user:
    return False

user_email = token_data['email']
user._password_hash, user._salt = user._hash_password(new_password)
self.user_repo.update(user)
...

```

Why it's better: The original code was not just insecure but **non-functional** `reset_password` tried to recover the user's email from `token.split('_')[-1]`, but `reset_password_request` never put the email in the token in the first place, so a real reset could never succeed (and the token-generation line crashes outright with `TypeError: 'int' object is not subscriptable`, since `len(...)` is being sliced). The fix uses `secrets.token_urlsafe(32)` (cryptographically random, unguessable) as an opaque key into a server-side `{token: {email, expires_at}}` map, exactly mirroring how a real reset-token table would work. It also adds the expiry check that was described in a comment but never implemented, and makes tokens single-use, closing a replay window. I verified this end-to-end: registering a user, requesting a reset, consuming the token, and logging in with the new password all now work, and a second use of the same token is correctly rejected.

3. Eliminate duplicated export-and-manifest logic

File: `eduplatform/services/export_service.py`

Original (this exact block appeared three times, once per method, only the manifest fields differed)

```

result = {}
for data_type, data in export_data.items():    # or class_data / export_data
    if not data:
        continue
    filename = f"{base_filename}_{data_type}.{format}"
    output_path = os.path.join(output_dir, filename)
    try:
        exported_path = self.export_utils.export_data(
            data=data, output_path=output_path, format=format,
            sheet_name=data_type.replace('_', ' ').title()
        )
        result[data_type] = exported_path
    except Exception as e:
        print(f"Warning: Failed to export {data_type}: {str(e)}")
        continue

```

```

manifest = {'export': {... method-specific fields ..., 'file_paths': result}}
manifest_path = os.path.join(output_dir, f"{base_filename}_manifest.json")
with open(manifest_path, 'w') as f:
    import json
    json.dump(manifest, f, indent=2)

result['manifest'] = manifest_path
return result

```

Improved

```

# --- new shared helpers, added once, right after __init__ ---
def _export_datasets(self, datasets, output_dir, base_filename, format):
    """Export each non-empty named dataset; used by all three export_* methods."""
    result = {}
    for data_type, data in datasets.items():
        if not data:
            continue
        filename = f"{base_filename}_{data_type}.{format}"
        output_path = os.path.join(output_dir, filename)
        try:
            result[data_type] = self.export_utils.export_data(
                data=data, output_path=output_path, format=format,
                sheet_name=data_type.replace('_', ' ').title()
            )
        except Exception:
            logger.warning("Failed to export dataset '%s'", data_type, exc_info=True)
    return result

def _write_manifest(self, output_dir, base_filename, manifest_body):
    manifest_path = os.path.join(output_dir, f"{base_filename}_manifest.json")
    with open(manifest_path, 'w') as f:
        json.dump({'export': manifest_body}, f, indent=2)
    return manifest_path

# --- each export_* method now ends with just: ---
result = self._export_datasets(export_data, output_dir, base_filename, format)
result['manifest'] = self._write_manifest(output_dir, base_filename, {
    'user_id': user_id, 'user_name': user_full_name, 'timestamp': timestamp,
    'exported_data': list(result.keys()), 'file_paths': result
})
return result

```

Why it's better: This exact ~25-line block was copy-pasted three times in the same file Pylint's **duplicate-code** check never caught it (confirmed by running it in isolation: it scored the file

10/10) because each copy used a differently-named local dict (`export_data` vs. `class_data`), which defeats the checker's line-level matching. Any future change to the export/error-handling behavior (e.g., retry logic, or writing failures into the manifest) previously had to be made in three places and was easy to miss in one. Now there's exactly one place that knows how to "export a set of datasets and write a manifest," and the three public methods just supply their own data and manifest fields. This also removed the `import json` statement duplicated inside three different function bodies. File length dropped from 425 to 377 lines with no behavior change (verified by compiling and by tracing through the logic).

4. Eliminate duplicated grade-serialization logic

File: `eduplatform/services/grade_service.py`

Original (identical 14-field dict built in two different methods)

```
result = []
for grade in grades:
    teacher = self.user_repo.get(grade._teacher_id)
    result.append({
        'id': grade._id, 'subject': grade._subject, 'type': grade._type.value,
        'score': grade._score, 'max_score': grade._max_score,
        'percentage': grade.percentage, 'letter_grade': grade.letter_grade,
        'gpa_points': grade.gpa_points, 'comments': grade._comments,
        'teacher_id': grade._teacher_id,
        'teacher_name': teacher._full_name if teacher else 'Unknown',
        'assignment_id': grade._assignment_id,
        'created_at': grade._created_at.isoformat(),
        'updated_at': grade._updated_at.isoformat() if grade._updated_at else None
    })
```

Improved

```
# new helper, added once, right after __init__
def _serialize_grade(self, grade: Grade) -> Dict[str, Any]:
    """Convert a Grade into the dict shape used by every grade-listing endpoint."""
    teacher = self.user_repo.get(grade._teacher_id)
    return {
        'id': grade._id, 'subject': grade._subject, 'type': grade._type.value,
        'score': grade._score, 'max_score': grade._max_score,
        'percentage': grade.percentage, 'letter_grade': grade.letter_grade,
        'gpa_points': grade.gpa_points, 'comments': grade._comments,
        'teacher_id': grade._teacher_id,
```



```

        'teacher_name': teacher._full_name if teacher else 'Unknown',
        'assignment_id': grade._assignment_id,
        'created_at': grade._created_at.isoformat(),
        'updated_at': grade._updated_at.isoformat() if grade._updated_at else None
    }

```

get_student_grades now just does:

```
result = [self._serialize_grade(grade) for grade in grades]
```

get_class_grades now just does:

```
student_grades = [self._serialize_grade(grade) for grade in grades]
```

Why it's better: Same root cause as #3 two call sites, one concept ("what a grade looks like to a client"), duplicated instead of shared. Before this change, adding a field to grade output (e.g. a rubric link) meant remembering to update it in two places; missing one silently made `get_student_grades` and `get_class_grades` return inconsistent shapes for the same underlying data. Now there is a single source of truth, and both methods are shorter and clearly about "get the right grades" rather than "get the right grades *and* know how to serialize a Grade."

5. Replace the hardcoded "25 students per class" magic value

File: `eduplatform/services/assignment_service.py`

Original

```

result.append({
    ...
    'total_submissions': total_submissions,
    'graded_submissions': graded_submissions,
    'submission_rate': (total_submissions / 25) * 100, # Assuming 25 students per class
    ...
})

```

Improved

inside the loop, before building the dict:

```
class_size = self._get_class_size(assignment._class_id)
```

```

...
result.append({
    ...
    'total_submissions': total_submissions,
    'graded_submissions': graded_submissions,

```

```

        'submission_rate': (total_submissions / class_size * 100) if class_size else None,
        ...
    })

# new helper, added near _calculate_average_grade:
def _get_class_size(self, class_id: str) -> int:
    """Get the number of students actually enrolled in a class."""
    return sum(1 for student in self.user_repo.get_students() if student.grade == class_id)

```

Why it's better: The old code silently produced a wrong number for every class that wasn't exactly 25 students a 30-student class at 100% submission would report "120%", a 10-student class could never exceed 40%. This wasn't a style nit; it was a real, invisible correctness bug that no linter flags because `25` is an ordinary integer literal. The fix computes the real roster size from `UserRepository.get_students()`, matching each student's `grade` (their class label, e.g. `"7-B"`) against the assignment's `_class_id`. I verified this against synthetic data (3 students in `"7-B"`, 1 in `"8-A"`) and got the correct counts (3 and 1, respectively, not a fixed 25). It also guards the empty-roster case by returning `None` instead of dividing by zero or reporting a meaningless rate.